



CONTROL DE VERSIONES

Autor(es)	Fecha de modificación	Versión	Descripción del cambio	Revisó	Estado
VSSL	02/12/2024	1.0	Creación del Documento	IIB	Revisado
VSSL	09/12/2024	1.1	Modificación del Documento	----	Pendiente

Propósito

El manual de mantenimiento proporciona una guía para garantizar el correcto funcionamiento y mantenimiento del proyecto desarrollado en Trabajo Terminal 2, abarcando tanto la aplicación móvil como los servicios web asociados. Este documento está diseñado para que desarrolladores, técnicos y personal de mantenimiento comprendan las herramientas utilizadas en el desarrollo y las tecnologías implementadas.

La aplicación desarrollada permite a los usuarios capturar fotografías o seleccionar imágenes desde la galería, identificando automáticamente el tipo de ave y mostrando información relevante sobre esta, a partir de ello la aplicación permite realizar bitácoras y muestreos para una organización de cada sesión de observación de aves. Adicionalmente se desarrolló una página web donde se visualizan las bitácoras y muestreos realizados por los usuarios.

El documento describe los elementos clave que pueden ser objeto de mantenimiento, como servidor de backend, aplicación móvil, dependencias externas, y bases de datos. También se proporciona una lista detallada de los productos utilizados en el desarrollo, junto con sus características y ubicaciones.

Este documento tiene como permite apoyar las actividades de mantenimiento preventivo, correctivo y evolutivo, asegurando la continuidad operativa y la adaptabilidad del proyecto a futuras necesidades.



Listado de dependencias.

Producto	Descripción
<u>Aplicación Móvil</u>	Ubicación: Código fuente disponible en el repositorio privado de GitHub: App_IdBird .
Kotlin, Java, XML	Lenguajes de programación utilizados para desarrollar la aplicación móvil. Kotlin y Java implementan la lógica de operación y las funcionalidades críticas, mientras que XML se emplea para el diseño de interfaces de usuario.
TensorFlow Lite (v2.14.0)	Frameworks y Dependencias de la Aplicación Móvil. Implementación de modelos de redes neuronales (Yolo v8s y MobileNet v3 large), permite la identificación y clasificación de aves en imágenes seleccionadas o tomadas con la cámara. Dataset utilizado: El modelo fue entrenado con un conjunto de datos de 15,000 imágenes de aves provenientes de INaturalist y eBird, siendo 3,000 imágenes por ave, optimizadas para su uso con TensorFlow Lite en dispositivos móviles.
Google Play Services Location (v21.0.1)	Frameworks y Dependencias de la aplicación Móvil. Obtención precisa de coordenadas de ubicación, lo que permite registrar la ubicación de cada bitácora o muestreo.
Glide (v4.15.1)	Frameworks y Dependencias de la Aplicación Móvil. Gestión de carga y visualización de imágenes en la aplicación móvil, como las fotos tomadas o seleccionadas por el usuario.
Room (v2.6.1)	Frameworks y Dependencias de la Aplicación Móvil. Es la base de datos local utilizada en la aplicación para cada dispositivo móvil. Manejo de base de datos local SQLite para almacenamiento local.
Retrofit2 (v2.9.0)	Frameworks y Dependencias de la Aplicación Móvil. Cliente HTTP para comunicación y para realizar consultas al API. Permite enviar y recibir datos mediante solicitudes RESTful, como sincronización de bitácoras y autenticación de usuarios.
<u>API de Backend</u>	Código fuente disponible en el repositorio privado de GitHub: API de correos .
Node.js (v20.17.0)	Framework principal de la API, desarrollado en JavaScript. Maneja la lógica de operación, los endpoints RESTful y la



	interacción con la base de datos.
MongoDB Atlas	Base de datos en la nube utilizada por la API para almacenar usuarios, aves, bitácoras y registros de muestreo. La conexión se realiza a través de Mongoose.
Bcrypt (v5.1.1)	Dependencias de la API Utilizado para realizar el hashing seguro de tokens al momento del registro y autenticación de usuarios, además de manejar el acceso a rutas permitidas al usuario.
Bootstrap (v5.3.3)	Dependencias de la API Librería CSS para diseño de interfaces web.
CORS (v2.8.5)	Dependencias de la API Control de acceso entre orígenes para seguridad. Para gestionar la seguridad de acceso a la API y permitir solicitudes desde orígenes diferentes a los del servidor.
Dotenv (v16.4.5)	Dependencias de la API Manejo de variables de entorno en el backend. Para manejar variables de entorno (claves, rutas de la base de datos) de forma segura almacenadas en el archivo .env.
EJS (v3.1.10)	Dependencias de la API Motor de plantillas para la renderización de vistas dinámicas en el servidor.
Express (v4.19.2)	Dependencias de la API Framework central de la API, empleado para manejar rutas y peticiones HTTP. Define los endpoints utilizados por la aplicación móvil y la página web.
JSONWebToken (JWT) (v9.0.2)	Dependencias de la API Manejo de autenticación segura con tokens. Para implementar autenticación segura mediante tokens, lo que garantiza que solo usuarios autorizados accedan a funcionalidades protegidas.
Mongoose (v8.6.0)	Dependencias de la API ODM para interactuar con MongoDB Atlas. Permitiendo operaciones CRUD en las colecciones de usuarios, bitácoras y registros.
Morgan (v1.10.0)	Dependencias de la API Registro de peticiones HTTP en el servidor. Middleware implementado para registrar todas las solicitudes HTTP recibidas, facilitando la depuración y monitoreo.
PostageApp (v2.0.0)	Dependencias de la API Servicio externo utilizado en la API para el envío automático de correos electrónicos, como los correos de confirmación



	para el registro de usuarios.
Serve-favicon (v2.5.0)	Dependencias de la API Middleware utilizado para gestionar y servir íconos estáticos, optimizando las respuestas HTTP.
Nodemon (v3.1.7)	Dependencias de la API Herramienta utilizada en el entorno de desarrollo de la API para reiniciar automáticamente el servidor al detectar cambios en el código.

Características mínimas del sistema

Aplicación Móvil

- Requisitos de Hardware:
 - o Procesador: Snapdragon serie 700, Mediatek Dimensity serie 800/900, o equivalentes (Octa-core, hasta 2.4 GHz).
 - o Memoria RAM: Mínimo 4 GB.
 - o Almacenamiento Interno: Al menos 79.9 MB libres para la instalación inicial, además de almacenamiento de imágenes o bitácoras.
 - o Cámara: Gran angular con resolución mínima de 13 MP (f/1.8) con enfoque automático y buena iluminación.
- Requisitos de Software:
 - o Sistema Operativo: Android 10 o superior
 - o Conectividad: Acceso a internet para sincronización de datos y uso de APIs externas.

Servidor de Backend

- Requisitos del Servidor:
 - o Procesador: CPU multinúcleo.
 - o Memoria RAM: 2 GB mínimo.
 - o Almacenamiento: 71.6 MB mínimo
- Software Necesario:
 - o Node.js: Versión 20.17.0 o superior.
 - o NPM para gestión de dependencias.
 - o Conectividad: Acceso a internet para sincronización de datos y uso de APIs externas.



Mantenimientos.

El mantenimiento del sistema tiene como objetivo garantizar el correcto funcionamiento de la aplicación móvil y la página web.

1. Mantenimiento Preventivo

El mantenimiento preventivo se enfoca en evitar fallos futuros, manteniendo el sistema actualizado y optimizado. Las actividades incluyen actualización de dependencias y revisión de configuraciones.

Recomendaciones:

- Antes de aplicar cambios, siempre trabajar en una rama de pruebas y ejecutar pruebas unitarias e integrales.
- Realizar un cronograma de mantenimiento preventivo, dependiendo del nivel de actividad y uso del sistema, siguiendo el formato de Bitácora de Mantenimiento más adelante propuesto.
- Implementar herramientas de monitoreo para automatizar tareas como el respaldo de datos y el análisis de seguridad.

API

- Actualización Segura de Dependencias

Antes de actualizar dependencias, ejecutar pruebas unitarias e integrales para detectar posibles fallos.

Evitar actualizaciones mayores sin revisar cambios en la documentación oficial.

1. Crear una rama de mantenimiento: `git checkout -b maintenance_InicialesAutor_actualizacionDependencias`
 2. Usar `npm outdated` para identificar dependencias obsoletas: `npm outdated`
 3. Actualizar dependencias menores y probar: `npm update`
 4. Ejecutar pruebas con Postman o Talend API Tester y validar compatibilidad.
- Monitoreo de Registros de la API
 1. Revisar periódicamente los logs generados desde Render o herramientas de monitoreo.
 2. Buscar errores 4xx/5xx recurrentes y tomar acciones correctivas.
 - Limpieza de logs y bases de datos.

Se sugiere revisar si existen logs excedentes o que sean innecesarios para el monitoreo de los productos, además de revisar si existen registros en la base de datos inservibles o que generen problemas.

1. Revisar los logs generados en las consolas de render o herramientas de



monitoreo.

2. Identificar los logs excedentes y eliminarlos o remplazarlos conforme se necesite.
3. Revisar los registros en las tablas de la base de datos e identificar datos excedentes o que no cumplan con el estándar de registro.
4. Eliminar o modificar los registros deseados.
5. Ejecutar pruebas con Postman o Talend API Tester y validar el funcionamiento correcto.

- **Respaldo de Base de Datos**

Se recomienda crear respaldos periódicamente para mantener segura la información y muestreos realizados por los usuarios, con el fin de evitar un daño a la base de datos y sus registros.

1. Realizar respaldos automáticos de MongoDB Atlas utilizando las configuraciones de backup del proveedor.
2. Guardar los respaldos en un espacio seguro.

- **Verificación de Seguridad (CORS y XSS)**

1. Validar las configuraciones de seguridad en la API (CORS) para evitar solicitudes no autorizadas.
2. Validar uso JSON y middlewares de autenticación de token.
3. Comprobar el correcto uso de tokens y acceso a las rutas permitidas al usuario y las permitidas a desarrolladores.

Aplicación Móvil

- **Actualización Segura de Dependencias**

Antes de actualizar dependencias, ejecutar pruebas unitarias e integrales para detectar posibles fallos.

Evitar actualizaciones mayores sin revisar cambios en la documentación oficial.

1. Crear una rama de mantenimiento: `git checkout -b maintenance_InicialesAutor_actualizacionDependencias`
2. Actualizar dependencias en build.gradle para evitar obsolescencia
3. Ejecutar pruebas de la aplicación en distintos dispositivos y versiones de Android.

- **Pruebas de Compatibilidad de Dispositivos**

1. Realizar pruebas en varios emuladores y dispositivos físicos con versiones de Android 10.0+.
2. Herramientas recomendadas: emuladores de Android Studio.

- **Monitoreo del Uso de Memoria y Rendimiento**



1. Utilizar Android Profiler en Android Studio para detectar fugas de memoria (memory leaks) y problemas de rendimiento en actividades críticas.
2. Optimizar código donde sea necesario.
- Optimización de Imágenes y Recursos
 1. Comprimir imágenes si es posible y recomendado en la carpeta `Drawable`.
 2. Revisar tamaños de imágenes para mejorar tiempos de carga.

Página Web

- Revisión de Librerías Frontend
 1. Crear una rama de mantenimiento: `git checkout -b maintenance_InicialesAutor_actualizacionDependencias`
 2. Actualizar librerías como Bootstrap y EJS asegurando compatibilidad.
 3. Ejecutar pruebas con Postman o Talend API Tester y validar compatibilidad.
- Optimización de Archivos Estáticos
 1. Comprimir imágenes y archivos CSS/JS en la carpeta `Static/`.
 2. Revisar tamaños de imágenes para mejorar tiempos de carga y mantener la calidad.
- Pruebas de Carga y Rendimiento
 1. Utilizar herramientas como Postman o Talend API Tester para simular múltiples usuarios accediendo al sistema.
 2. Identificar posibles cuellos de botella y optimizar consultas.

2. Mantenimiento Correctivo

El mantenimiento correctivo se aplica cuando se detectan errores en el sistema. Su objetivo es solucionar fallos puntuales para restaurar el funcionamiento normal.

API

- Error en el Inicio del Servidor
La API no se inicia.
 1. Validar que el servicio de Render esté activo.
 2. Reiniciar el servidor manualmente desde Render.



- Fallo en la Conexión con MongoDB Atlas

El servidor no puede acceder a la base de datos.

1. Revisar la cadena de conexión en el archivo .env:
DB_URI=mongodb+srv://usuario:contraseña@cluster.mongodb.net/db
2. Verificar que el servicio de MongoDB Atlas esté activo.
3. Verificar que la IP esté permitida en MongoDB Atlas.

- Token JWT Inválido

Las solicitudes protegidas devuelven código de error 0003 validación incorrecta, 0009 token invalido o 0010 token inexistente, o el error 404 Not Found.

1. Confirmar la clave secreta en el archivo .env:
JWT_SECRET=clave_secreta_segura
2. Validar que el token sea correcto y no esté expirado.
3. En caso de ser necesario crear otro token o modificar la clave.

- Error al Enviar Correos con PostageApp

Los correos no se envían. Muestra un error parecido a { "response" : { "uid" : "27cf6ede7501a32d54d22abe17e3c154d2cae7f3", "status" : "bad_request", "message" : "Some error message. Like: you forgot the recipients!" } }

1. Validar la clave de API en el archivo .env.
2. Comprobar que cada elemento del correo se esté declarando y enviando correctamente.
3. Probar la solicitud con Postman para identificar problemas adicionales.

Aplicación Móvil

- Detección y Corrección de Errores:

1. Abrir Android Studio y revisar el Logcat para identificar los errores.
2. Realizar cambios en el código según la causa del fallo.
3. Volver a compilar y ejecutar la aplicación en un emulador o dispositivo físico.
4. Validar que el error haya sido solucionado.

Ejemplo:

Si se presenta un fallo al obtener la ubicación, revisar la implementación de Google Play Services Location y actualizar permisos en el AndroidManifest.xml.

- Reestablecer Aplicación Corrupta

1. Limpiar caché y datos desde Configuración del dispositivo → Aplicaciones → Limpiar caché.
2. Desinstalar y reinstalar la aplicación.
3. Validar que las configuraciones y sincronización con la API funcionen correctamente.



- Crash de la Aplicación al Procesar el Modelo de TensorFlow Lite

La app se cierra al analizar imágenes.

1. Confirmar que los modelos `model.tflite` y `best_float32-tflite` estén en la carpeta `Assets`.
2. Validar la ruta de carga de los modelos en el código.
3. Confirmar que el funcionamiento de carga y preprocesamiento de la imagen sea correcto.
4. Confirmar que el modelo de identificación de aves (`best_float32.tflite`) este funcionando correctamente.
5. Confirmar que el modelo de clasificación de aves (`model.tflite`) este funcionando correctamente.
6. Depurar usando Android Studio y revisar el Logcat para identificar los errores.
7. Realizar cambios en el código según la causa del fallo.
8. Volver a compilar y ejecutar la aplicación en un emulador o dispositivo físico.
9. Validar que el error haya sido solucionado.

Página Web

- Error al Renderizar Vistas (EJS)

La página muestra un error o 404 Not found o muestra la pantalla de error en vez de mostrar la pantalla de bitacoras o muestreos.

1. Revisar la ruta de la carpeta `Views`.
2. Validar que el nombre del archivo coincida con `maps.ejs` y `bitacora.ejs`

- Problemas de Responsividad

La página no se adapta a dispositivos móviles.

1. Revisar archivos `CSS` y aplicar `Media Queries`.
2. Recargar la página.

- Registros no cargan

La página no muestra ningún muestreo o bitacoras, dependiendo de en que página se encuentre el usuario.

1. Revisar la conexión correcta en la base de datos.
2. Revisar que existan registros de bitácoras y muestreos en la base de datos de MongoDB Atlas.
3. Realizar cambios en el código según la causa del fallo.
4. Recargar la página.



3. Mantenimiento Evolutivo

El mantenimiento evolutivo permite añadir nuevas funcionalidades al sistema, mientras que el mantenimiento adaptativo ajusta el sistema para que siga funcionando correctamente en nuevos entornos.

API

- Adaptación a Nuevos Requisitos:
 1. Crear una nueva ruta o actualizar una existente en la carpeta Routes.
 2. Implementar la lógica correspondiente en la carpeta Controllers.
 3. Actualizar el modelo si se requieren nuevos campos en la base de datos.
 4. Probar los cambios utilizando Postman y validar la integración con la aplicación móvil.

Ejemplo:

Adaptar la API para permitir filtrado avanzado de bitácoras mediante parámetros query.

- Actualizar Node.js y Dependencias
 1. Descargar e instalar la última versión de Node.js desde nodejs.org
 2. Actualizar las dependencias globalmente: `npm install -g npm@latest`
 3. Probar el servidor con la nueva versión: `npm run dev`

- Migrar a una Nueva Base de Datos (si es necesario)

Migrar de MongoDB Atlas a una solución alternativa:

1. Exportar los datos actuales: `mongodump --uri 'mongodb+srv://Vania_10:Vania_10@idbird.psrme.mongodb.net/?retryWrites=true&w=majority&appName=IdBird'`
2. Importar los datos a la nueva base: `mongoexport --uri mongodb://nueva_base_de_datos`

Aplicación Móvil

- Incorporación de Nuevas Funcionalidades:
 1. Crear una nueva rama en el repositorio (Git): `git checkout -b feature/nueva-funcionalidad`
 2. Implementar la funcionalidad en el código.
 3. Probar el cambio localmente en el emulador o dispositivo físico.
 4. Realizar un pull request para fusionar los cambios con la rama principal.

Ejemplo:

Añadir una funcionalidad para compartir las bitácoras registradas por un enlace.



- Compatibilidad con Nuevos Dispositivos
 1. Realizar pruebas en dispositivos recientes utilizando herramientas como Firebase Test Lab.
 2. Ajustar tamaños y resoluciones en la carpeta res/ Drawable, para garantizar adaptabilidad.

Página Web

- Actualizar a una Nueva Versión de Bootstrap
 1. Descargar la última versión desde getbootstrap.com.
 2. Reemplazar los enlaces en las vistas Pages:
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
- Ajustar el Diseño para Navegadores Modernos
 1. Probar la página con herramientas como BrowserStack para verificar compatibilidad.
 2. Aplicar cambios en el diseño si hay errores de renderización.

Elementos que pueden recibir mantenimiento.

Aplicación Móvil:

- Interfaz de usuario:
 - o Mejorar la experiencia de usuario (UX) mediante ajustes en el diseño y navegación.
 - o Actualizar iconos, estilos y flujos de interacción para cumplir con estándares actuales.
- Módulo de Identificación de Aves:
 - o Actualización de los modelos de redes neuronales, MobileNet, para incluir más especies.
 - o Mejorar la precisión del reconocimiento bajo condiciones de baja iluminación o imágenes borrosas.
- Gestión de Bitácoras:
 - o Mejorar la funcionalidad para búsqueda y filtrado de registros.
- Compatibilidad con Dispositivos:
 - o Asegurar el correcto funcionamiento en nuevas versiones de Android y actualizaciones de hardware.

API de Backend:



- Autenticación y Seguridad:
 - Implementar mejores prácticas en la configuración de CORS y HTTPS.
 - Implementar más seguridad y claves para las consultas.
- Rutas y Endpoints:
 - Agregar nuevos endpoints para funcionalidades adicionales.
- Conexión con la Base de Datos:
 - Agregar nuevas tablas si se crean más.
- Servidor de Backend:
 - Actualizar Node.js y sus dependencias para compatibilidad y rendimiento.
 - Implementar escalabilidad horizontal en caso de aumento en el tráfico de usuarios.

Base de Datos:

- Carga de Datos:
 - Actualizar la base de datos con nuevas especies de aves y datos relevantes.
 - Realizar migraciones de datos si se modifican las estructuras existentes.
- Rendimiento:
 - Escalar el clúster de MongoDB Atlas si aumenta el volumen de datos o usuarios.

Integraciones con APIs Externas:

- PostageApp:
 - Revisar y actualizar las configuraciones para envío de correos electrónicos.
 - Agregar plantillas personalizadas para nuevas notificaciones.



Plan de Respuesta Ante Fallos

Caída de la API

Síntomas:

- El cliente móvil o la página web no puede conectarse a la API.
- Se reciben errores HTTP 500 (Internal Server Error).

Protocolo de Respuesta:

1. Detección del Problema:
 - El cliente reporta el fallo al desarrollador a través de los canales definidos (correo, herramienta de gestión de incidencias como Jira).
 - Responsable: Cliente o personal de soporte técnico.
2. Diagnóstico Inicial:
 - Verificar si el servidor está activo en Render.
 - Revisar los logs del servidor para identificar la causa del fallo
 - Responsable: Desarrollador Backend.
3. Acciones Correctivas:
 - Si el servidor está detenido, reiniciarlo a través de Render o manualmente desplegarlo.
 - Si los logs indican un fallo en una dependencia o archivo faltante:
 - Verificar el entorno con `npm install`.
 - Reinstalar dependencias si es necesario: `npm install`
 - Responsable: Desarrollador Backend.
4. Validación del Servicio:
 - Probar los endpoints críticos con Postman o Talend API Tester para asegurarse de que la API esté operativa.
 - Responsable: Desarrollador Backend o Equipo de QA.
5. Notificación:
 - Informar al cliente que el servicio ha sido restaurado.
 - Responsable: Desarrollador Backend o Soporte Técnico.

Eliminación de la Base de Datos

Síntomas

- Los datos no están disponibles o la API devuelve errores relacionados con la conexión a la base de datos, como errores 500 o de código 0002 o 0004.

Protocolo de Respuesta

1. Detección del Problema:
 - El sistema de monitoreo (como MongoDB Atlas Dashboard) o el cliente notifica la ausencia de datos.



- Responsable: Cliente o Administrador de la Base de Datos.
- 2. Diagnóstico Inicial:
 - Revisar si la base de datos está activa desde el panel de control de MongoDB Atlas.
 - Verificar la conectividad desde el servidor:
mongo
"mongodb+srv://Vania_10:Vania_10@idbird.psrme.mongodb.net/?retryWrites=true&w=majority&appName=IdBird" --username usuario
 - Responsable: Desarrollador Backend o Administrador de la Base de Datos.
- 3. Protocolo de Recuperación:
 - Si la base de datos fue eliminada:
 - Restaurar un respaldo reciente desde MongoDB Atlas:
 1. Acceder a la opción Backups en el panel de MongoDB Atlas.
 2. Seleccionar el respaldo más reciente y restaurar.
 - Confirmar que la conexión de la API funciona correctamente
 - Responsable: Administrador de la Base de Datos.
 - Si no hay respaldos disponibles:
 - Notificar al cliente sobre la pérdida irreversible de datos.
 - Responsable: Desarrollador Backend o Administrador de la Base de Datos
- 4. Validación:
 - Realizar consultas de prueba a la base de datos para asegurar que los datos han sido restaurados.
 - Probar endpoints dependientes de la base de datos utilizando Postman.
 - Responsable: Desarrollador Backend o Equipo de QA.
- 5. Notificación:
 - Informar al cliente que los datos han sido restaurados.
 - Responsable: Desarrollador Backend o Administrador de la Base de Datos.

La Aplicación No Sincroniza los Datos

Síntomas

- Los datos de bitácoras y muestreos no se envían a la API, o la aplicación muestra un mensaje de error al intentar sincronizar.
- En los logs aparece un error HTTP (404 o 500).

Protocolo de Respuesta



1. Detección del Problema:
 - El cliente reporta que la sincronización no funciona correctamente.
 - Responsable: Cliente o usuario final.
2. Diagnóstico Inicial:
 - Capturar los logs en Android Studio para identificar el problema:
 - Revisar la configuración de la URL base de la API:

```
val retrofit =  
Retrofit.Builder().baseUrl("https://idbird-api.onrender.com")  
.addConverterFactory(GsonConverterFactory.create()).build()
```
 - Confirmar que la API esté activa.
 - Responsable: Desarrollador Backend y Móvil.
3. Corrección:
 - Ajustar la URL de la API en caso de errores de configuración.
 - Revisar la implementación de las solicitudes HTTP en Retrofit:

```
@POST("/bitacoras/verificarActualizarConMuestreos")  
fun subirBitacoraConMuestreos(@Body BitacoraConMuestreos): Call<RespuestaRegistro>
```

payload:
 - Responsable: Desarrollador Móvil.
4. Validación:
 - Probar la sincronización con datos reales en un emulador o dispositivo físico.
 - Validar que la API responde correctamente.
 - Responsable: Equipo de QA.
5. Notificación:
 - Notificar al cliente que el problema ha sido resuelto.
 - Responsable: Desarrollador Móvil.

Imágenes No Cargan Correctamente

Síntomas

- Los usuarios reportan que las fotos tomadas con la cámara no aparecen en la aplicación.
- En los logs se encuentra un error relacionado con FileNotFoundException o permisos.

Protocolo de Respuesta

1. Detección del Problema:
 - Revisar los permisos en la configuración del dispositivo.
 - Responsable: Usuario final.
2. Diagnóstico Inicial:
 - Capturar los logs de la aplicación con Android Studio.
 - Validar si los permisos necesarios están solicitados en el archivo



AndroidManifest.xml:

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission
  android:name="android.permission.READ_MEDIA_IMAGES" />
<uses-permission
  android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

- Responsable: Desarrollador Móvil.

3. Corrección:

- Solicitar permisos antes de acceder a la cámara o galería.
- Validar que las rutas de las imágenes sean correctas.
- Responsable: Desarrollador Móvil.

4. Validación:

- Probar la funcionalidad en diferentes dispositivos y versiones de Android.
- Responsable: Desarrollador Móvil o Equipo de QA.

5. Notificación:

- Publicar una actualización de la aplicación corrigiendo el problema.
- Responsable: Desarrollador Móvil.



Formato de tabla para Bitácoras de Mantenimientos

Se propone este formato de tabla para llevar la bitácora de mantenimientos realizados. Este formato permite registrar información clave, como quién realizó el mantenimiento, la parte del sistema afectada, y el protocolo seguido.

Bitácora de Mantenimiento			
Fecha:	[DD-MM-YYYY]	Responsable:	[Nombre del desarrollador o equipo encargado del mantenimiento.]
Parte del Sistema:	[Aplicación Móvil / API / Página Web]	Tipo de Mantenimiento:	[Preventivo / Correctivo / Evolutivo]
Descripción del problema/mejora:		[Breve descripción del problema o la mejora implementada. Ejemplo: "Actualización de dependencias de Retrofit en la app móvil".]	
Protocolo seguido:		[Describir el procedimiento realizado para resolver el problema o implementar la mejora. Ejemplo: "Se ejecutó npm update y pruebas con Postman".]	
Observaciones:		[Observaciones o comentarios relevantes para el cambio]	
Resultado:		[Éxito / Pendiente / Falló. Ejemplo: "Éxito, sin errores detectados".]	